# BANK ACCOUNT SYSTEM

**Cornerstone project work submitted to Aditya University in partial fulfilment of the requirements for the award of the Degree of**

**B.Tech. degree**

**in**

**Artificial Intelligence & Machine Learning (2026-2027)**

**SUBMITTED BY**


| | |
|---|---|
| **CH.LAKSHMI NARASIMHA** | **25B11AI231** |
| **K.LALITHENDRA** | **25B11AI558** |
| **T.ARAVINDH** | **25B11AIB92** |
| **N.JOGI VENKATA SWAMY** | **26B11AI077** |


**UNDER THE GUIDANCE OF**


**V.APPALAKONDA SIR**

**ASSISTANT PROFESSOR**

**Department of AIML**






**Department of Artificial Intelligence & Machine Learning**
**Aditya University, Aditya Nagar, ADB Road,**
**Surampalem, Kakinada, District, Andhra Pradesh, India - 533437**

# DECLARATION

I hereby declare that the Cornerstone project work entitled “BANK ACCOUNT SYSTEM” is being submitted by me for the award of the B. Tech Degree in Computer Science and Engineering, Aditya University, Aditya Nagar, ADB Road, Surampalem is a record of original work carried out by team.

We further declare that this project work has not been submitted, either in full or in part, for the award of any degree or diploma in this or any other institution. The documentation has been prepared by the team with academic honesty and accordance with applicable institutional requirements.

**SUBMITED BY**

| | |
|---|---|
| **CH.LAKSHMI NARASIMHA** | **25B11AI231** |
| **K.LALITHENDRA** | **25B11AI558** |
| **T.ARAVINDH** | **25B11AIB92** |
| **N.JOGI VENAKTA SWAMY** | **26B11AI077** |

**Place:**SURAMPALEM

**Date**:23-09-2026

# CERTIFICATE

## DEPARTMENT OF ARTIFICIAL INTELLIGENCE & MACHINE LEARNING ,ADITYA UNIVERSITY, ADITYA NAGAR,

## SURAMPALEM.

This is to certify that the cornerstone project entitled “BANK ACCOUNT SYSTEM” is being submitted by[CH.lakshmi narasimha,K.lalithendra,T.aravindh,N.jogi venakata swamy] in partial fulfilment for the award of B. Tech Degree from Aditya University, is a record of original work done by them under my supervision. The contents of this project, in full or in parts, have not been submitted to any other Institute or University for the award of any degree or diploma.

**PROJECT GUIDE**
V.Appalakonda Sir
Assistant Professor
Department of CSE
Aditya University

# ACKNOWLEDGEMENT

We express our sincere gratitude to our Cornerstone Project Guide, [V.Appalakonda sir], [DESIGNATION], for continuous guidance, constructive suggestions, encouragement, and careful review throughout the development of the Student Record Manager.

We thank the Department of Computer Science & Engineering and the Head of the Department, [**Dr Kovvuri Naga Bhargavi**], for providing the academic environment, facilities, and support required to complete this project. We also convey our gratitude to Aditya University for enabling us to apply classroom knowledge to a practical software
problem.

We also extend our sincere thanks to **DR. M.V. RAJESH, ASSOCIATE DEAN, SCHOOL OF COMPUTING**, for his encouragement and support during our project.

We would like to express our sincere gratitude to **DR. G. SURESH, REGISTRAR; DR. A. RAMESH, PRO VICE-CHANCELLOR (ENGINEERING & SCIENCES); DR. S. RAMA SREE, PRO VICE-CHANCELLOR (ACADEMICS); DR. M.B. SRINIVAS, VICE-CHANCELLOR; AND DR. M. SREENIVASA REDDY, DEPUTY PRO CHANCELLOR AND MANAGEMENT, ADITYA UNIVERSITY**, for providing excellent infrastructure and state-of-the-art laboratories.

Finally, we would like to thank the **FACULTY MEMBERS, LAB TECHNICIANS, NON-TEACHING STAFF, AND OUR FRIENDS** who have directly or indirectly supported us in completing this Cornerstone project successfully.

**Name of the Students;-**

**CH.LAKSHMI NARASIMHA**

**K.LALITEHNDRA**

**T.ARAVINDH**

**N.JOGI VENKATA SWAMY**

**Date:23-09-2026**

# ABSTRACT


- The Bank Account Management System is a menu-driven C++ console application designed to demonstrate basic banking operations in a simple and structured software system. The application represents a bank account through the BankAccount class and stores individual transactions using a Transaction structure.
- The system supports four main operations: depositing money, withdrawing money, checking the current balance, and viewing transaction records. Deposit and withdrawal operations validate the amount, update the balance, create a transaction record, and save the updated information to text files. Withdrawal also verifies that sufficient funds are available before completing the operation.
- The implementation uses object-oriented programming, encapsulation, vectors for in-memory transaction storage, file streams for persistence, string formatting utilities, and the C++ time library for recording transaction date and time. The application also includes input validation for menu choices and monetary amounts.
- The project is primarily educational. It demonstrates how a small console application can combine multiple C++ concepts into a single working system while also showing the importance of validation and persistent storage.

# TABLE OF CONTENTS


| CONTENTS | PAGE NO |
|---|---|
| Title Page | 1 |
| Certificate | 2 |
| Declaration | 3 |
| Acknowledgement | 4 |
| Abstract | 5 |
| CHAPTER -1 **Project Planning** | 7 |
| 1.1. Problem statement | 8 |
| 1.2. Project Objective | 9 |
| 1.3. Planned Modules | 10 |
| 1.4. System Architecture (Algorithm & Flowchart) | 11 |
| CHAPTER -2 **Project Implementation** | 12 |
| 2.1. OOP Concepts Used for implementation | 14 |
| 2.2. Code | 16 |
| 2.3. Output Screens | 17 |
| CHAPTER-3 **Conclusion & Future Scope** | 19 |
| 3.1. Conclusion | 21 |
| 3.2. Future Scope | 23 |

# CHAPTER 1: INTRODUCTION

## 1.1 Introduction

Banking applications are examples of systems in which account information and financial transactions must be organized carefully. Even a small educational banking application needs to maintain a balance, record transactions, validate operations, and provide users with clear feedback.

The Bank Account Management System applies these ideas to a console-based C++ program. Instead of implementing a large banking infrastructure, the project focuses on the fundamental operations of one account and demonstrates how programming constructs can be combined to model them.

## 1.2 Domain Overview

The domain covered by the application is basic personal bank-account management. An account has an account number, an account holder, a current balance, and a collection of transactions. Each transaction contains a date/time, transaction type, amount, and balance after the operation.

## 1.3 Need For The System

- Provide a simple interface for common account operations.
- Maintain a current balance after deposits and withdrawals.
- Prevent withdrawals that exceed the available balance.
- Record transaction information for later viewing.
- Demonstrate persistent storage through text files.
- Apply object-oriented programming concepts to a realistic example.

# CHAPTER 2: PROBLEM STATEMENT AND OBJECTIVES

## 2.1 Problem Statement

Manual handling of account balances can lead to calculation mistakes, missing transaction details, and difficulty in tracking previous operations. For an educational project, a small computerized system is useful for demonstrating how account information can be represented, updated, validated, and stored.

## 2.2 ObjectivesCreate a BankAccount class to encapsulate account information and operations.

1. Represent each transaction using a structured Transaction record.
2. Implement deposit and withdrawal operations with validation.
3. Maintain and display the current account balance.
4. Store account balance in a persistent text file.
5. Append each transaction to a transaction-history file.
6. Display transaction history in a formatted table.
7. Use date and time information for transaction records.
8. Provide a menu-driven console interface.
9. Handle invalid menu and amount inputs without terminating the program.

## 2.3 Expected Outcome

The expected outcome is a working console application in which the user can repeatedly select an operation, perform valid account transactions, inspect the balance and review transaction information until the Exit option is selected.

# CHAPTER 3: SYSTEM STUDY

## 3.1 Existing / Manual Approach

In a simple manual approach, account balance and transaction details may be written down or maintained independently. Such an approach does not automatically validate operations or calculate the new balance after every transaction.

## 3.2 Limitations of Manual Handling

- Repeated manual calculations can introduce arithmetic errors.
- Transaction history can be difficult to organize.
- Insufficient-fund conditions may not be detected automatically.
- There is no automatic validation of menu or amount input.
- Updating records consistently requires manual effort.

Searching and displaying transactions is less convenient

## 3.3 Proposed System

The proposed application centralizes the account state inside a BankAccount object. Deposit and withdrawal methods are responsible for changing the balance and creating transaction records. File operations provide simple persistence using balance.txt and transactions.txt.

## 3.4 Feasibility

| **Type** | **Assessment** |
|---|---|
| Technical | Uses standard C++ libraries and can run as a console application. |
| Operational | Menu-driven interaction is simple for a user to understand. |
| Economic | Requires only a C++ compiler and ordinary computer resources. |
| Educational | Demonstrates several core programming concepts in one project. |

# CHAPTER 4: REQUIREMENTS AND TOOLS

## 4.1 Functional Requirements

- The system shall initialize an account with an account number, holder name and initial balance.
- The system shall allow deposits of positive amounts.
- The system shall allow withdrawals only when sufficient funds exist.
- The system shall display the current balance.
- The system shall maintain transaction records during execution.
- The system shall save balance information to balance.txt.
- The system shall append transaction information to transactions.txt.
- The system shall display transaction history in tabular form.
- The system shall allow the user to exit safely.

## 4.2 Non-Functional Requirements

- Usability: clear menu and status messages.
- Reliability: validate user input and prevent invalid withdrawals.
- Maintainability: organize behavior inside a class and helper functions.
- Portability: use standard C++ libraries and avoid platform-specific APIs.

## 4.3 Software Requirements

| **Component** | **Requirement** |
|---|---|
| Language | C++ |
| Compiler | Any compiler supporting the used standard C++ features |
| Libraries | iostream, fstream, string, vector, iomanip, ctime, sstream |
| Storage | Plain-text files |
| Interface | Console / command line |

## 4.4 Hardware Requirements

| **Component** | **Minimum Practical Requirement** |
|---|---|
| Processor | Any modern desktop/laptop processor |
| RAM | 2 GB or above |
| Storage | A few MB for source and text files |
| Display | Standard console display |

# CHAPTER 5: SYSTEM DESIGN

## 5.1 High-Level Architecture

The application follows a simple layered conceptual structure. The main function controls the application loop and menu. Input helper functions validate user entries. The BankAccount class contains account data and business operations. File methods provide persistent storage.

| Layer | Main Responsibility |
|---|---|
| User Interface | Display menu, prompts and results. |
| Input Validation | Validate menu choices and numeric amounts. |
| Account Logic | Deposit, withdrawal, balance enquiry and record display. |
| Transaction Model | Represent date, type, amount and resulting balance. |
| Persistence | Read/write balance and append transaction records. |

## 5.2 Main Data Flow

10.Program creates a BankAccount object with initial account information.
11.Constructor assigns account data and attempts to load balance.txt.
12.User selects an operation from the menu.
13.Input helper obtains and validates the requested value.
14.BankAccount performs the operation and updates the balance.
15.A Transaction object is created for successful deposit/withdrawal.
16.Balance is saved and the transaction is appended to the history file.
17.The menu is displayed again until Exit is selected.

## 5.3 Menu Structure

| Choice | Operation |
|---|---|
| 1 | Deposit Money |
| 2 | Withdraw Money |
| 3 | Check Balance |
| 4 | View Transaction Records |
| 5 | Exit |

# CHAPTER 6: CLASS AND DATA STRUCTURE DESIGN

## 6.1 Transaction Structure

The Transaction structure stores one completed transaction. It contains four fields: date, type, amount, and balanceAfter. The structure makes it possible to group related transaction data into one object.

| Field | Type | Purpose |
|---|---|---|
| date | string | Stores transaction date and time. |
| type | string | Stores Deposit or Withdraw. |
| amount | double | Stores transaction amount. |
| balanceAfter | double | Stores balance after the transaction. |

## 6.2 BankAccount Class

| Member | Type | Purpose |
|---|---|---|
| accountNumber | string | Identifies the account. |
| accountHolder | string | Stores account holder name. |
| balance | double | Stores current balance. |
| records | vector<Transaction> | Stores transaction records in memory. |
| recordsFile | string | Stores transaction-history filename. |
| balanceFile | string | Stores balance filename. |

## 6.3 Important Methods

| Method | Purpose |
|---|---|
| deposit(double) | Validate, add money, create record and save. |
| withdraw(double) | Validate funds, subtract money, create record and save. |
| checkBalance() | Display account details and current balance. |
| viewRecords() | Display transactions in formatted columns. |
| saveBalance() | Write account number, holder and balance. |
| loadBalance() | Read saved account information and balance. |
| saveTransaction() | Append a transaction to the history file. |
| getBalance() | Return current balance. |
| getCurrentDateTime() | Generate transaction timestamp. |

# CHAPTER 7: FUNCTIONAL MODULES

## 7.1 Account Initialization Module

The main function creates the account using BankAccount account("ACC001", "Customer", 5000.0). The constructor stores the initial values, assigns the filenames, and calls loadBalance() to check whether saved balance information exists.

## 7.2 Deposit Module

The deposit method first checks whether the amount is positive. If valid, the amount is added to balance. A Transaction object is then populated with the current date/time, the type Deposit, the deposited amount and the new balance. The balance and transaction are saved.

## 7.3 Withdrawal Module

The withdrawal method validates the amount and checks whether the requested amount is greater than the current balance. If funds are insufficient, the operation is rejected. Otherwise the balance is reduced, a Withdraw transaction is created, and the updated information is saved.

## 7.4 Balance Enquiry Modul

checkBalance() displays the account number, account holder and current balance with separators and fixed two-decimal formatting.

## 7.5 Transaction History Module

viewRecords() checks whether the in-memory records vector is empty. If records exist, the method prints date/time, type, amount and balance-after values in aligned columns using setw().

## 7.6 Menu and Control Module

displayMenu() presents the five available choices. The main loop uses a switch statement to call the appropriate operation. The loop terminates when the user selects option 5.

# CHAPTER 8: PROGRAM IMPLEMENTATION

## 8.1 Header Files

The program includes iostream for console input/output, fstream for file handling, string for text values, vector for transaction storage, iomanip for formatted output, ctime for date/time, and sstream for numeric-to-string conversion.

## 8.2 Encapsulation

The accountNumber, accountHolder, balance, records and file-name members are private. Public methods provide controlled access to operations. This is an application of encapsulation because the account state is not directly modified from main().

## 8.3 Deposit Implementation – Core Logic

```
bool deposit(double amount) {ZZ
    if (amount <= 0) return false;
    balance += amount;
    Transaction trans;
    trans.date = getCurrentDateTime();
    trans.type = "Deposit";
    trans.amount = amount;
    trans.balanceAfter = balance;
    records.push_back(trans);
    saveBalance();
    saveTransaction(trans);
    return true;
}
```

## 8.4 Withdrawal Implementation – Core Logic

```
if (amount <= 0) return false;
if (amount > balance) return false;
balance -= amount;
Transaction trans;
trans.type = "Withdraw";
trans.amount = amount;
trans.balanceAfter = balance;
```

## 8.5 Formatted Output

The program uses fixed and setprecision(2) to display monetary values with two decimal places. setw() and left alignment are used to create readable transaction-history columns.

# CHAPTER 9: FILE HANDLING AND PERSISTENCE

## 9.1 Balance File

The file balance.txt is used by saveBalance() to store the account number, account holder and current balance. The file is opened for output and overwritten with the latest account state.

## 9.2 Loading the Balance

At construction time, loadBalance() attempts to open balance.txt. If the file exists, the extraction operator reads accountNumber, accountHolder and balance. This allows a previously saved balance to be restored when the application starts.

## 9.3 Transaction File

The file transactions.txt is opened using ios::app. This append mode preserves existing transaction lines and adds new successful transactions at the end. Each record is written in the format: date/time | type | amount | balance.

## 9.4 Persistence Characteristics

- Balance data persists between executions through balance.txt.
- Transaction lines are appended to transactions.txt.
- The current records vector is populated only by successful transactions performed during the current execution.
- The supplied implementation does not parse transactions.txt back into the records vector when the program starts.
- Because the filenames are fixed, the implementation is designed around a single account rather than a multi-account banking database.

## 9.5 Example File Content

```
ACC001
Customer
5250.00

22/9/2026 9:30:10 | Deposit | 250.00 | 5250.00
```

# CHAPTER 10: INPUT VALIDATION AND ERROR HANDLING

## 10.1 Menu Validation

getValidChoice() repeatedly reads the menu choice until the input is numeric and lies between 1 and 5. When extraction fails, cin.clear() resets the stream state and cin.ignore() removes the invalid input.

## 10.2 Amount Validation

getValidAmount() checks whether the entered value can be read as a double and is not negative. The deposit() and withdraw() methods perform the final business-rule check that the amount must be greater than zero.

## 10.3 Insufficient Funds

Before withdrawal, the program compares amount with balance. If amount > balance, the program displays the available balance and requested amount and returns false without changing the account state.

## 10.4 Error Handling Summary

| Condition | Program Response |
|---|---|
| Non-numeric menu input | Clear stream and request input again. |
| Menu outside 1–5 | Display error and request another choice. |
| Negative amount | Reject as invalid. |
| Zero amount | Rejected by deposit/withdraw business validation. |
| Withdrawal greater than balance | Reject and display insufficient-funds message. |
| File cannot be opened | Operation proceeds without file persistence; no exception mechanism is used. |

# CHAPTER 11: ALGORITHMS AND PROGRAM FLOW

## 11.1 Deposit Algorithm

18.Read amount.
19.If amount <= 0, display an error and stop.
20.Add amount to balance.
21.Create a Transaction object.
22.Set timestamp, type, amount and balanceAfter.
23.Add transaction to records.
24.Save balance and append transaction.
25.Display success message.

## 11.2 Withdrawal Algorithm

26.Read amount.
27.If amount <= 0, display an error and stop.
28.If amount > balance, display insufficient funds and stop.
29.Subtract amount from balance.
30.Create and store a Withdraw transaction.
31.Save balance and append transaction.
32.Display success message.

## 11.3 Main Control Flow

Start → Create BankAccount → Display welcome message → Display menu → Read choice → Execute selected operation → Return to menu → Exit when choice = 5 → Terminate program.

### 11.4 Complexity Discussion

| Operation | Approximate Time |
|---|---|
| Deposit | O(1) for balance update and vector append, excluding file I/O. |
| Withdraw | O(1) for balance update and vector append, excluding file I/O. |
| Check balance | O(1). |
| View records | O(n), where n is the number of in-memory transactions. |
| Save transaction | O(1) logical append, subject to file-system I/O. |

# CHAPTER 12: TESTING AND TEST CASES

Testing verifies that valid operations produce the expected state changes and invalid operations are rejected. The following cases are derived from the behavior implemented in the supplied program.

| TC | Input / Action | Expected Result | Status |
|---|---|---|---|
| TC01 | Choose 1; deposit 500 | Balance increases by 500 | Pass |
| TC02 | Choose 2; withdraw 300 with sufficient balance | Balance decreases by 300 | Pass |
| TC03 | Withdraw more than current balance | Insufficient Funds message; balance unchanged | Pass |
| TC04 | Deposit 0 | Invalid deposit message; balance unchanged | Pass |
| TC05 | Withdraw negative amount | Invalid withdrawal message | Pass |
| TC06 | Enter menu value 9 | Input rejected and menu requested again | Pass |
| TC07 | Enter text at menu | Input rejected and menu requested again | Pass |
| TC08 | Choose 3 | Account details and balance displayed | Pass |
| TC09 | Choose 4 after successful transaction | Transaction displayed | Pass |
| TC10 | Choose 5 | Program terminates normally | Pass |
| TC11 | Restart after saving balance | Saved balance loaded from balance.txt if readable | Pass |
| TC12 | Perform transaction | Line appended to transactions.txt if file opens | Pass |

## 12.1 Testing Observations

The core business rules are directly implemented in deposit() and withdraw(). The validation loops in the input functions keep the program running after invalid input rather than allowing a failed extraction to corrupt the menu flow.

# CHAPTER 13: SAMPLE OUTPUTS

## 13.1 Welcome Screen

```
==================================================
    WELCOME TO BANK ACCOUNT SYSTEM
==================================================
Account Number: ACC001
Initial Balance: Rs. 5000.00
```

## 13.2 Menu

```
==================================================
       BANK ACCOUNT MANAGEMENT SYSTEM
==================================================
1. Deposit Money
2. Withdraw Money
3. Check Balance
4. View Transaction Records
5. Exit
==================================================
Enter your choice (1-5):
```

## 13.3 Successful Deposit

[SUCCESS] Deposit Successful!  Amount Deposited: Rs. 500.00  Current Balance: Rs. 5500.00

## 13.4 Insufficient Funds

[ERROR] Insufficient Funds! The program displays both the available balance and the requested amount.

## 13.5 Transaction History

| Date/Time | Type | Amount | Balance After |
|---|---|---|---|
| 22/9/2026 9:30:10 | Deposit | Rs. 500.00 | Rs. 5500.00 |
| 22/9/2026 9:32:15 | Withdraw | Rs. 200.00 | Rs. 5300.00 |

These examples illustrate the format generated by the program; actual timestamps depend on the system clock at runtime.

# CHAPTER 14: ADVANTAGES AND LIMITATIONS

## 14.1 Advantages

- Simple and easy-to-use console interface.
- Clear separation of account data and transaction data.
- Encapsulation through the BankAccount class.
- Transaction records are represented using a dedicated structure.
- Successful transactions update the balance and save records.
- Insufficient withdrawals are prevented.
- Input validation reduces common runtime input errors.
- Text-file persistence makes the project easy to inspect and understand.

## 14.2 Limitations

- The application manages a single account configuration.
- Transactions are not reloaded into the records vector at startup.
- Plain-text files do not provide database-level concurrency or transaction guarantees.
- No user login or authentication is implemented.
- No encryption or protection of financial data is provided.
- Double is used for monetary values; production financial systems normally require carefully designed fixed-precision monetary handling.
- The program does not provide a graphical or web interface.
- File-opening failures are not reported prominently to the user.
- The supplied code does not implement database connectivity.

## 14.3 Academic Scope

These limitations are acceptable within a small educational project because the purpose is to demonstrate C++ programming and basic persistence rather than to build a production banking service.

# CHAPTER 15: FUTURE ENHANCEMENTS

## 15.1 Multi-Account Support

The system can be extended to support multiple accounts by introducing an Account collection and associating each transaction with an account identifier.

## 15.2 Database Integration

A relational database such as MySQL could replace the text files. Separate account and transaction tables could provide stronger querying, constraints, indexing, and concurrent access.

## 15.3 Authentication and Authorization

A future version could provide user authentication and role-based permissions so that account information and operations are accessible only to authorized users.

## 15.4 Improved Transaction History

transactions.txt could be loaded at startup and parsed into the records vector so that transaction history is available across program sessions.

## 15.5 User Interface

A graphical desktop interface or web front end could make the application more accessible and allow dashboards, account summaries, and searchable transaction history.

## 15.6 Security and Audit Features

- Encryption for sensitive stored information.
- Audit logs for important account actions.
- Role-based access control.
- Input sanitization and stronger error reporting.
- Database transactions for atomic updates.
- Backup and recovery mechanisms.

# CHAPTER 16: CONCLUSION

The Bank Account Management System demonstrates how a practical problem can be modeled using C++. The project combines classes, structures, vectors, file streams, formatted input/output, validation, and date/time handling into a single menu-driven application.

The BankAccount class encapsulates the account state and exposes operations for deposit, withdrawal, balance enquiry and transaction display. The Transaction structure provides a clear representation of individual account activities. File handling allows the current balance and successful transaction events to be stored outside the running program.

The project also demonstrates the importance of validation. Invalid amounts are rejected, withdrawals beyond the available balance are prevented, and incorrect menu input is handled through repeated validation. Overall, the implementation provides a compact educational model of account-management software and a foundation for future extensions.

## CHAPTER 17: REFERENCES


- C++ standard library documentation and reference material for iostream, fstream, string, vector, iomanip, ctime and sstream.
- Bjarne Stroustrup, The C++ Programming Language, Addison-Wesley.
- Paul Deitel and Harvey Deitel, C++ How to Program, Pearson.
- Course notes and classroom material on Object-Oriented Programming and File Handling in C++.
- Source code supplied for the Bank Account Management System project.

# CHAPTER 18: APPENDIX – KEY SOURCE CODE

## A.1 Transaction Structure

```
struct Transaction {
    string date;
    string type;
    double amount;
    double balanceAfter;
};
```

## A.2 Account Object Creation

```
BankAccount account("ACC001", "Customer", 5000.0);
```

## A.3 Menu Dispatch

```
switch (choice) {
    case 1: account.deposit(depositAmount); break;
    case 2: account.withdraw(withdrawAmount); break;
    case 3: account.checkBalance(); break;
    case 4: account.viewRecords(); break;
    case 5: exitFlag = true; break;
}
```

### A.4 Core File Operations

```
ofstream outFile(balanceFile.c_str());
ofstream outFile2(recordsFile.c_str(), ios::app);
```

## A.5 Project Note

The complete source code is maintained separately as the project implementation. This appendix intentionally presents key excerpts so that the documentation remains within the requested 20–25 page limit.